

02_CONCURRENCY: MUTUAL EXCLUSION AND SYNCHRONIZATION

1. INTRODUZIONE ALLA CONCORRENZA

Il design di un OS si occupa della gestione di processi e thread:

- Multiprogramming
- Multiprocessing
- Distributed Processing

Il problema dell'accesso alla stessa risorsa da parte di thread e processi vari, quindi della concorrenza, sorge da 3 risorse:

- applicazioni multiple, permette di condividere processing time fra diverse applicazioni attive
- applicazioni strutturate, estensione del modular design e structured programming
- struttura di OS, gli OS stessi sono implementati come un insieme di processi o thread

Avremo bisogno che l'OS gestisca la concorrenza.

Vediamo alcuni termini utili riguardanti la concorrenza

Operazione atomica: operazione indivisibile, non può quindi essere interrotta da nessun altro processo.

Sezione critica: porzione di codice di un processo che accede a delle risorse a cui possono accedere altri thread o altri processi (risorse condivise), e che deve non essere eseguita mentre un altro processo è in una corrispondente sezione del codice.

Mutua esclusione: quando un processo è in una sezione critica, nessun altro processo può essere in una sezione critica che accede ad alcuna di quelle risorse condivise

Race condition: molteplici thread o processi leggono e scrivono dati condivisi e il risultato finale dipende dal relativo tempo di esecuzione (chi arriva prima)

Deadlock: due o più processi non possono procedere perché ognuno sta aspettando che uno degli altri faccia qualcosa

Livelock: due o più processi cambiano continuamente il loro stato a causa di variazioni in altri processi, facendo quindi lavoro inutile

All'esame comparirà solo deadlock, non livelock

Busy waiting: è uno spreco di risorse, sono in attesa di un evento che tanto occorrerà più avanti. Ad esempio se so che un evento si verifica ogni 5 minuti, è uno spreco di risorse se rimango in attesa tutti i 5 minuti invece di passare il controllo a qualcun altro

Starvation: un processo runnabile non viene preso in considerazione indefinitamente dallo scheduler. Anche se è capace di proseguire, non viene mai scelto

Quando si parla di multiprogramming bisogna che l'output sia indipendente dalla velocità di esecuzione di altri processi concorrenti.

Principi della concorrenza:

- Interleaving (esecuzione in modo alternato o concorrente) e overlapping
Entrambi rappresentano lo stesso problema
- (Uniprocessor) La relativa velocità di esecuzione dei processi non può essere prevista, dipende da attività di altri processi, da come l'OS gestisce le interrupt e dalla politica di scheduling dell'OS

Difficoltà della concorrenza:

- Condivisione di risorse globali
- È difficile per l'OS gestire l'allocazione di risorse ottimalmente
- È difficile localizzare errori di programmazione in quanto i risultati non sono deterministici e riproducibili

La race condition avviene quando molteplici processi o thread leggono e scrivono dati, e il risultato finale dipende dall'ordine di esecuzione.

Il "perdente" della corsa è il processo che aggiorna per ultimo, e determinerà il valore finale della variabile.

Riguardo questioni di design e management causati dalla presenza di concorrenza, l'OS deve:

- essere in grado di tenere traccia di vari processi
- allocare e deallocare risorse per ogni processo attivo
- proteggere i dati e le risorse fisiche di ogni processo contro interferenze da altri processi
- assicurarsi che i processi e gli output sono indipendenti dalla velocità di processing

Interazione fra processi:

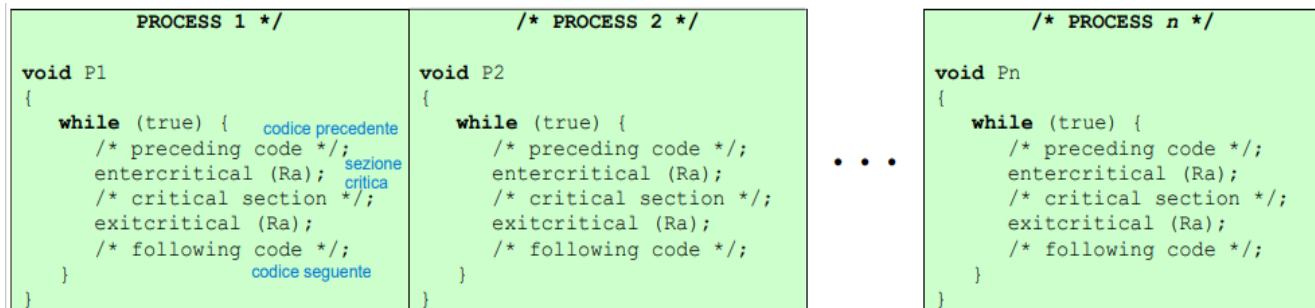
Degree of Awareness	Relationship	Influence that One Process Has on the Other	Potential Control Problems
Processes unaware of each other	Competition	• Results of one process independent of the action of others • Timing of process may be affected	• Mutual exclusion • Deadlock (renewable resource) • Starvation
Processes indirectly aware of each other (e.g., shared object)	Cooperation by sharing	• Results of one process may depend on information obtained from others • Timing of process may be affected	• Mutual exclusion • Deadlock (renewable resource) • Starvation • Data coherence
Processes directly aware of each other (have communication primitives available to them)	Cooperation by communication	• Results of one process may depend on information obtained from others • Timing of process may be affected	• Deadlock (consumable resource) • Starvation

Processi concorrenti entrano in conflitto quando sono in competizione per l'utilizzo della stessa risorsa, ad esempio I/O devices, memory, processor time, clock.

Nel caso di competing processes bisogna affrontare 3 problemi di controllo:

- bisogno di mutua esclusione
- deadlock
- starvation

es.



Si vuole gestire l'ingresso e uscita nella sezione critica. All'ingresso si vogliono bloccare altri processi che vogliono accedere alla stessa risorsa (non quelli che accedono ad altre).

La mutua esclusione dev'essere imposta.

Un processo che si ferma in una sua sezione non critica deve farlo senza interferire con altri processi

Non si deve avere deadlock o starvation.

A un processo non deve essere negato l'accesso a una sezione critica quando non c'è nessun altro processo che la sta usando.

Non si deve fare nessuna assunzione riguardo relativa velocità di processo o numero di processi.

Un processo rimane dentro la sua sezione critica solo per un tempo finito.

Supporto hardware alla mutua esclusione:

Interrupt Disabling (uniprocessor system, si disabilitano gli interrupt), garantisce mutua esclusione: allow a process to disable interrupts before it enters its critical section and then enable interrupts after it leaves its critical section. By disabling interrupts the CPU will be unable to switch processes.

Svantaggi:

- l'efficienza dell'esecuzione potrebbe essere degradata notevolmente
- questo approccio non funziona in architetture multiprocessor

Compare&Swap Instruction (o compare and exchange instruction): vengono confrontati un valore in memoria con un valore di test. Se i valori sono uguali si esegue uno swap del valore in memoria con un nuovo valore.

Viene eseguito automaticamente. Implementazione mutua esclusione con Compare&Swap:

```
int compare_and_swap(int* reg, int oldval, int newval)
{
    ATOMIC();
    int old_reg_val = *reg;
    if (old_reg_val == oldval)
        *reg = newval;
    END_ATOMIC();
    return old_reg_val;
}
```

Se il valore puntato in *reg* è uguale al valore di test *oldval*, allora scrive *newval* in *reg*.

In ogni caso restituisce il valore che era inizialmente salvato in *reg* (che sia stato cambiato o meno).

```

/* program mutualexclusion */
const int n = /* number of processes */;
int bolt;
void P(int i)
{
    while (true) {
        while (compare_and_swap(&bolt, 0, 1) == 1)
            /* do nothing */;
        /* critical section */;
        bolt = 0;
        /* remainder */;
    }
}

void main()
{
    bolt = 0;
    parbegin (P(1), P(2), . . . , P(n));
}

```

variabile che uso per capire se qualcuno è nella sezione critica (0:libera, 1:occupata)

resta nel ciclo senza fare nulla finché bolt è =1
se bolt=0 (sezione critica è libera) ci scrivo 1 (entro io), la condizione è falsa, esce quindi dal ciclo

Quindi: appena vedo che bolt è 0, ci scrivo 1, entro nella sezione critica e alla fine delle operazioni scrivo di nuovo 0 in bolt per liberarla

bolt inizializzato a 0 (sezione critica libera)

n processi concorrenti

In ogni processo vi è quindi un busy waiting

Un meccanismo simile è il seguente

Exchange instruction: scambia il contenuto di un registro con quello di una memory location

```

void exchange (int *register, int *memory)
{
    int temp;
    temp = *memory;
    *memory = *register;
    *register = temp;
}

```

Scambia il valore salvato in register col valore salvato memory.

```

/* program mutualexclusion */
int const n = /* number of processes*/;
int bolt;
void P(int i)
{
    while (true) {
        int keyi = 1;
        do exchange (&keyi, &bolt) while (keyi != 0);
        /* critical section */;
        bolt = 0;
        /* remainder */;
    }
}

void main()
{
    bolt = 0;
    parbegin (P(1), P(2), . . . , P(n));
}

```

scambia il valore salvato in keyi (1) col valore in bolt finché keyi != 0

Se bolt=1 (sezione critica occupata):
scrive 1 in bolt e 1 in keyi, resta quindi nel ciclo.
Se bolt=0 (sezione critica libera):
scrive 1 in bolt (occupa la sezione critica) e 0 in keyi, esce quindi dal ciclo

sezione critica libera la sezione critica

Vantaggi di questi meccanismi:

- Applicabili a un qualsiasi numero di processi sia su un single processor sia su multiple processors che condividono una main memory
- Semplice e facile da verificare
- Può essere usato per supportare molteplici sezioni critiche, ognuna può essere definita dalla propria variabile

Svantaggi:

- Viene impiegato busy waiting: mentre un processo aspetta per avere l'accesso a una sezione critica continua a consumare processor time
- La starvation è possibile quando un processo lascia una sezione critica e più di un processo sta aspettando
- Deadlock è possibile
- Non disponibile in qualsiasi architettura (il developer dev'essere a conoscenza dell'architettura)

2. MECCANISMI DI CONCORRENZA: SEMAFORI, MONITOR, MESSAGE PASSING

Meccanismi di concorrenza comuni:

- **Semaforo**: un costrutto basato su un valore intero usato per mandare segnali fra processi. Su un semaforo si possono eseguire solo 3 operazioni atomiche: inizializzare, decrementare e incrementare.
L'operazione di decremento potrebbe causare il blocking di un processo, e l'operazione di incremento potrebbe causare l'unblocking di un processo
- **Semaforo binario**: semaforo che prende solo i valori di 0 e 1
- **Mutex**: simile a un semaforo binario. Una differenza è però che il processo che blocca il mutex (setta la variabile a 0) dev'essere quello che lo sblocca (setta il valore a 1)
- **Condition variable**: è un tipo di dato usato per bloccare un processo o un thread finché una particolare condizione non diventa vera
- **Monitor**: è la versione del semaforo in programmazione orientata agli oggetti. È un costrutto che incapsula variabili, procedure d'accesso e codice di inizializzazione in un tipo di dato astratto. La variabile del monitor può essere acceduta solamente attraverso la sua procedura d'accesso e solamente un processo per volta può accedere al monitor. Le procedure d'accesso sono sezioni critiche.
Un monitor potrebbe avere una coda di processi in attesa di accedervi.
- **Event flags**: una memory word viene usata come meccanismo di sincronizzazione. L'application code potrebbe associare un diverso evento con ciascun bit nella flag.
Un thread può aspettare che un singolo evento o una combinazione di eventi semplicemente controllando uno o molteplici bit nella flag corrispondente.
Il thread può essere bloccato finché tutti i bit richiesti sono settati (AND), o finché almeno uno dei bit viene settato (OR).
- **Mailboxes/messaggi**: è un modo per due processi di scambiare informazioni e che potrebbe essere usato per la sincronizzazione.
- **Spinlocks**: meccanismo di mutua esclusione in cui un processo esegue un ciclo infinito aspettando che il valore di una variabile di lock indichi disponibilità.

- **Semaforo**: variabile che ha un valore intero su cui sono definite solo 3 operazioni:
 - Inizializzazione: inizializza a un valore intero non negativo, si dichiarano cioè quante unità di risorsa sono disponibili
 - **semWait** (decrementa): un processo chiede di riservare un'unità della risorsa. Se non sono disponibili unità (dopo aver decrementato, il valore del semaforo è <0), il processo viene messo in **blocking**, cioè in attesa, per essere risvegliato solo quando gli sarà assegnata l'unità richiesta
 - **semSignal** (incrementa), un processo rilascia l'unità di risorsa, per la quale altri processi potrebbero già essere in attesa, causandone quindi l'**unblocking**

Un processo entra quindi in blocking quando richiede una risorsa occupata con la semWait.
Si verifica l'unblocking quando un altro processo rilascia una risorsa con la semSignal.

In teoria non ci dovrebbe essere modo di ispezionare o manipolare i semafori se non con queste operazioni.

Prima che un processo decrementi il semaforo non c'è modo di sapere se si bloccherà o no (prima di richiedere l'unità di risorsa non può sapere se questa è disponibile).

Su un sistema uniprocessor se ci sono due processi concorrenti, non c'è modo di sapere quale processo continuerà immediatamente dopo una semWait.

Non possiamo sapere se un altro processo sta aspettando, quindi il numero di processi unblocked potrebbe essere 0 o 1. Dopo averlo decrementato non posso sapere se c'era un processo in attesa o no.

es. semaforo

```
struct semaphore {
    int count;
    queueType queue;
};
void semWait(semaphore s)
{
    s.count--; richiede un unità di risorsa
    if (s.count < 0) { la risorsa non è disponibile, deve aspettare
        /* place this process in s.queue */;
        /* block this process */;
    }
}
void semSignal(semaphore s)
{
    s.count++; rilascia un unità di risorsa
    if (s.count <= 0) { c'era un processo bloccato in attesa, va messo in ready
        /* remove a process P from s.queue */;
        /* place process P on ready list */;
    }
}
```

es. semaforo binario

```

struct binary_semaphore {
    enum {zero, one} value;
    queueType queue;
};

void semWaitB(binary_semaphore s)
{
    if (s.value == one) risorsa libera
        s.value = zero; occupa la risorsa
    else { risorsa occupata, deve aspettare
        /* place this process in s.queue */;
        /* block this process */;
    }
}

void semSignalB(semaphore s)
{
    if (s.queue is empty()) non ci sono processi in attesa
        s.value = one; risorsa libera
    else { ci sono processi in attesa, va messo in ready
        /* remove a process P from s.queue */;
        /* place process P on ready list */;
    }
}

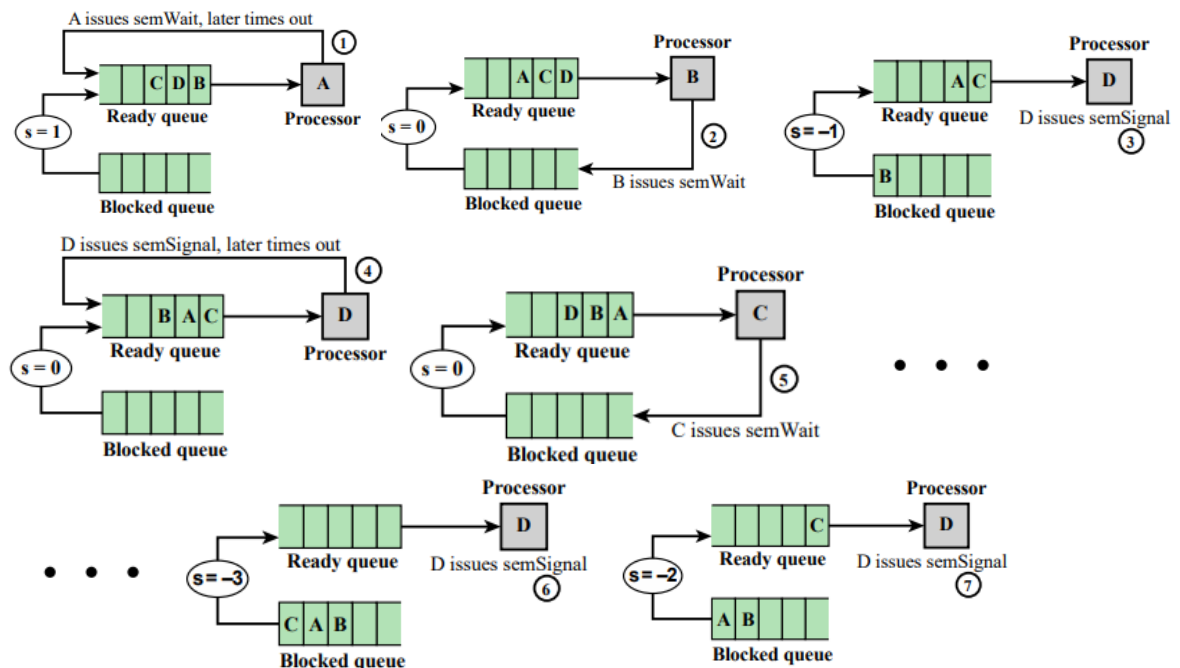
```

Una coda contiene i processi che aspettano il semaforo. Si ha una coda per ogni semaforo.

Semaforo strong: per scegliere quale processo fra i blocked vanno messi in ready si usa politica FIFO, evita starvation

Semaforo weak: non è specificato l'ordine in cui i processi vengono rimossi dalla coda

es. flusso di esecuzione con semaforo



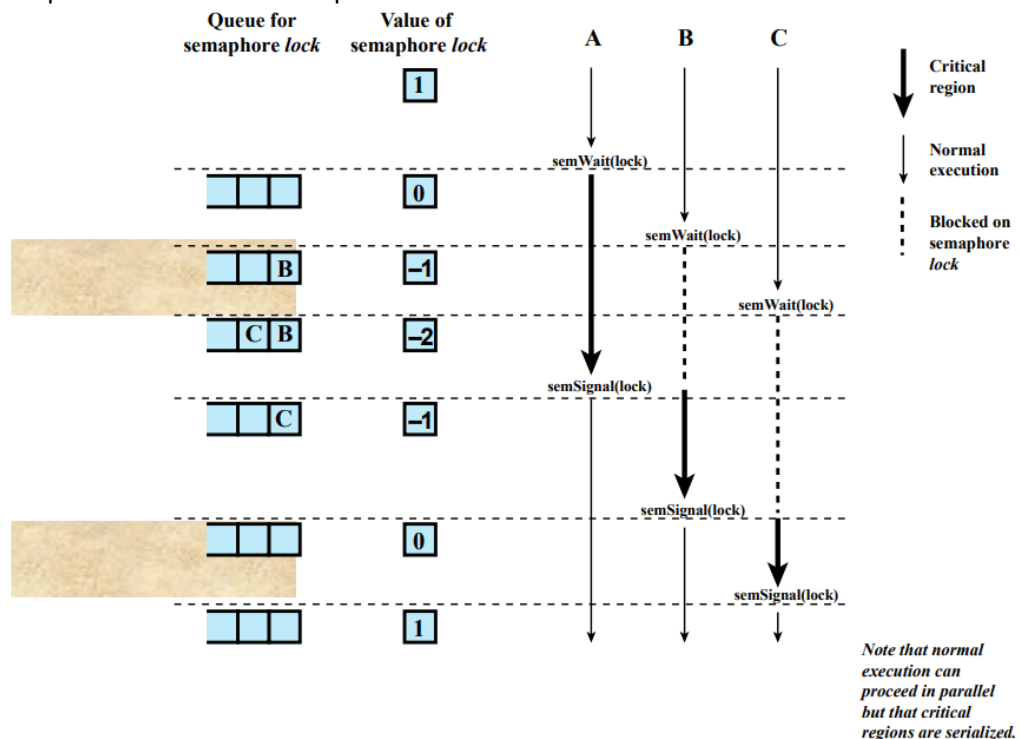
Mutua esclusione (unica risorsa) con semafori:

```

/* program mutualexclusion */
const int n = /* number of processes */;
semaphore s = 1; c'è un'unica risorsa
void P(int i)
{
    while (true) {
        semWait(s); richiede risorsa
        /* critical section */; sezione critica
        semSignal(s); rilascia risorsa
        /* remainder */;
    }
}
void main()
{
    parbegin (P(1), P(2), . . . , P(n));
}

```

Accesso di processi a dati condivisi protetti da un semaforo:



Implementazione dei semafori:

- È obbligatorio che la semWait e semSignal siano implementate come primitive atomiche, quindi due processi non possono eseguire una di queste primitive contemporaneamente (manipolare un semaforo è un problema di mutua esclusione)
- Può essere implementata in hardware o firmware
- Possono essere usati software schemes come gli algoritmi di Dekker's o Peterson's
- Si usa uno degli hardware-supported schemes per la mutua esclusione

Vediamo due possibili implementazioni di semaforo.

Implementazione con compare and swap:

```
semWait(s)
{
    while (compare_and_swap(s.flag, 0, 1) == 1)
        /* do nothing */;
    s.count--;
    if (s.count < 0) {
        /* place this process in s.queue */;
        /* block this process (must also set s.flag to 0) */;
    }
    s.flag = 0;
}

semSignal(s)
{
    while (compare_and_swap(s.flag, 0, 1) == 1)
        /* do nothing */;
    s.count++;
    if (s.count <= 0) {
        /* remove a process P from s.queue */;
        /* place process P on ready list */;
    }
    s.flag = 0;
}
```

semWait(s):

- `while (compare_and_swap(s.flag, 0, 1) == 1)`: **resta nel ciclo senza fare nulla finché s.flag=1** (qualcuno sta usando una primitiva del semaforo). Se s.flag=0 (sezione critica del semaforo libera) la condizione è falsa e quindi esce dal ciclo.
- `/* do nothing */;`: **flag all'interno del semaforo, serve per evitare che diversi processi entrino nella sezione critica del semaforo contemporaneamente** (le operazioni sono atomiche).
- `s.count--;`: **decrementa (richiede la risorsa)**.
- `if (s.count < 0)`: **se non ci sono risorse disponibili**.
 - `/* place this process in s.queue */;`: **mette il processo in coda (blocked)**.
 - `/* block this process (must also set s.flag to 0) */;`: **blocca il processo**.
- `s.flag = 0;`: **libera la sezione critica del semaforo**.

semSignal(s):

- `while (compare_and_swap(s.flag, 0, 1) == 1)`: **come sopra, aspetta che la sezione critica del semaforo sia libera, poi la occupa**.
- `/* do nothing */;`: **come sopra, aspetta che la sezione critica del semaforo sia libera, poi la occupa**.
- `s.count++;`: **incrementa (rilascia la risorsa)**.
- `if (s.count <= 0)`: **se c'era un processo blocked in attesa**.
 - `/* remove a process P from s.queue */;`: **rimuove un processo dalla coda e lo mette in ready**.
 - `/* place process P on ready list */;`: **rimuove un processo dalla coda e lo mette in ready**.
- `s.flag = 0;`: **libera la sezione critica del semaforo**.

Implementazione con interrupt:

```
semWait(s)
{
    inhibit interrupts;
    s.count--;
    if (s.count < 0) {
        /* place this process in s.queue */;
        /* block this process and allow interrupts */;
    }
    else
        allow interrupts;
}

semSignal(s)
{
    inhibit interrupts;
    s.count++;
    if (s.count <= 0) {
        /* remove a process P from s.queue */;
        /* place process P on ready list */;
    }
    allow interrupts;
}
```

semWait(s):

- `inhibit interrupts;`: **blocca gli interrupt (per l'atomicità)**.
- `s.count--;`: **decrementa (richiede la risorsa)**.
- `if (s.count < 0)`: **se non ci sono risorse disponibili**.
 - `/* place this process in s.queue */;`: **mette il processo in coda (blocked)**.
 - `/* block this process and allow interrupts */;`: **blocca il processo e permette gli interrupt**.
- `else allow interrupts;`: **se ci sono risorse disponibili, permette gli interrupt**.

semSignal(s):

- `inhibit interrupts;`: **blocca gli interrupt (per l'atomicità)**.
- `s.count++;`: **incrementa (rilascia la risorsa)**.
- `if (s.count <= 0)`: **se c'era un processo blocked in attesa**.
 - `/* remove a process P from s.queue */;`: **rimuove un processo dalla coda e lo mette in ready**.
 - `/* place process P on ready list */;`: **rimuove un processo dalla coda e lo mette in ready**.
- `allow interrupts;`: **permette gli interrupt**.

Semafori Posix:

- Inizializzazione: `int sem_init(sem_t * sem, int pshared, unsigned value);`
- Wait: `int sem_wait(sem_t *sem);`
- Signal: `int sem_post(sem_t *sem);`
- Distruzione: `int sem_destroy(sem_t *sem);`

Con parametri:

- *sem*: il semaforo
- *pshared*: 0 se il semaforo è condiviso fra threads, 1 se è condiviso fra processi (noi vedremo sempre il caso in cui è 0)
- *value*: il valore di inizializzazione (quante risorse possiamo condividere)

Tutte queste funzioni restituiscono -1 in caso di errore, altrimenti 0.

C'è una variabile "errno" che viene settata per identificare i tipi di errore

es. semaforo Posix

```
...
#include <semaphore.h>

sem_t sem;

void* thread_fun(void* arg){
    sem_wait(&sem);
    //critical section l'accesso alla sezione critica è gestito dal semaforo
    sem_post(&sem);
}

int main (){
    sem_init(&sem,0,1); //inizializza semaforo, condiviso fra thread, una sola risorsa disponibile
                        //(un solo thread per volta può accedere alla sezione critica)
    pthread_t t1,t2;
    pthread_create(&t1,NULL,thread,NULL);
    pthread_create(&t2,NULL,thread,NULL); //crea due thread
    pthread_join(t1,NULL);
    pthread_join(t2,NULL); //attende i due thread
    sem_destroy(&sem); //elimina il semaforo
    return 0;
}
```

Semafori in Java:

- Inizializzazione: `Semaphore(int value);`
`Semaphore(int value, boolean how);` //specifica il tipo di semaforo (weak o strong)
- Wait: `acquire();`
- Signal: `release();`

con parametri:

- *value*: valore di inizializzazione (numero di risorse che abbiamo)
- *How*: True (semaforo strong) o False (semaforo weak, scelta di default)

es. semaforo Java

```
...
import java.util.concurrent.*;

class MyThread extends Thread
{
    Semaphore sem;
    String threadName
    public MyThread(Semaphore sem, String threadName) {
        super(threadName);
        this.sem = sem;
        this.threadName = threadName;
    }
    public void run(){
        sem.acquire();
        //critical section
        sem.release();
    }
}

public class SemaphoreDemo{
    int main (){
        Semaphore sem = new Semaphore(1);
        MyThread mt1 = new MyThread(sem, "A");
        MyThread mt2 = new MyThread(sem, "B");

        mt1.start();
        mt2.start();

        mt1.join();
        mt2.join();
    }
}
```

Semafori in Python:

- Inizializzazione: **threading.Semaphore([value]);**
- Wait: **acquire([blocking]);**
- Signal: **release();**

con parametri:

- *value*: valore di inizializzazione (numero di risorse che abbiamo, se omesso di default è 1)
- *blocking*: True (default, il processo è bloccato se il valore corrente non è positivo) o False (la procedura restituisce True se il valore corrente è positivo, False altrimenti, non blocca)

es. semaforo Python

```
import threading
sem = threading.Semaphore()

def fun():
    while True:
        sem.acquire()
        # critical section
        sem.release()

t1 = threading.Thread(target = fun)
t1.start()
t2 = threading.Thread(target = fun)
t2.start()
```

- **Monitor:** è un costrutto di linguaggio di programmazione orientato agli oggetti, che fornisce funzionalità equivalenti a quelle dei semafori, ma più facili da controllare.

È implementato in diversi linguaggi di programmazione.

È stato implementato anche come una program library.

Moduli software consistente in una o più procedure, una sequenza di inizializzazione e dati locali.

Caratteristiche dei moduli:

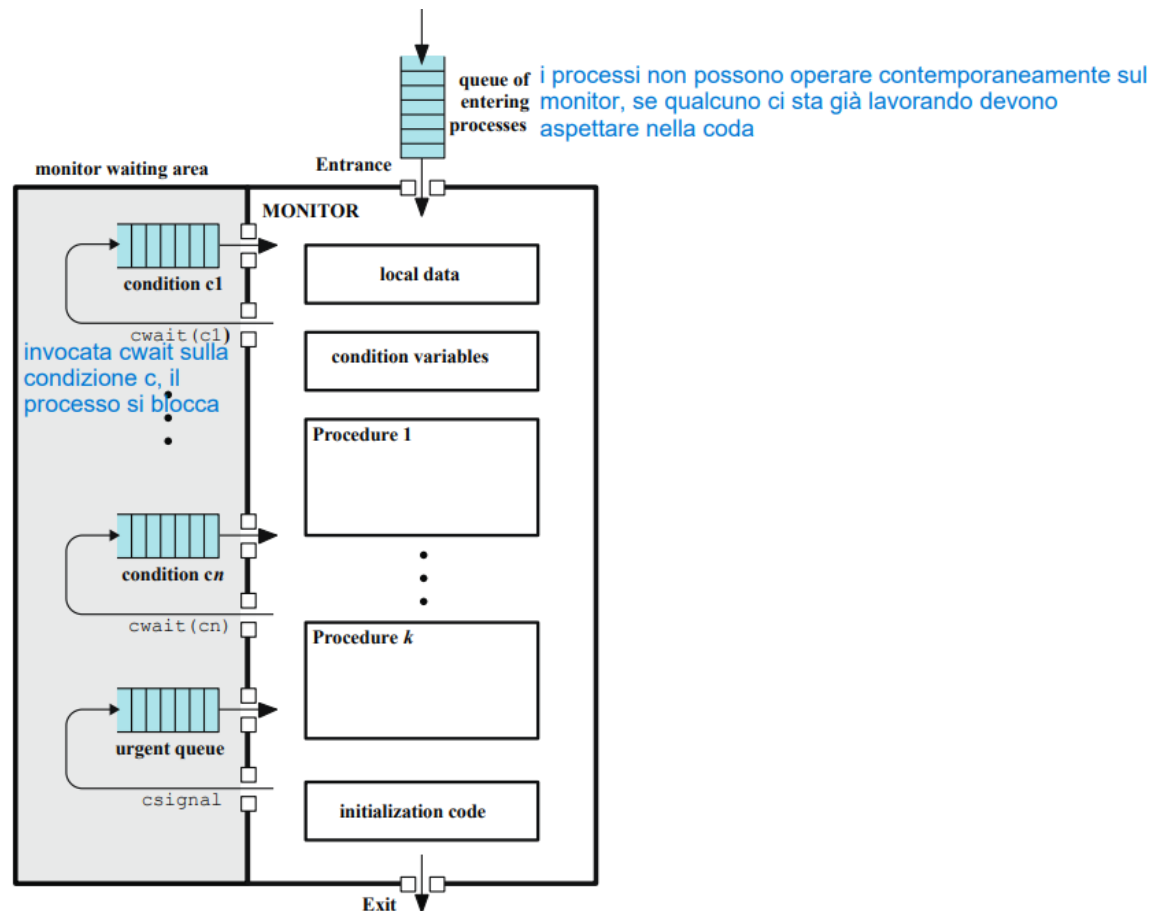
- Variabili di dati locali sono accessibili solo attraverso le procedure del monitor, e non attraverso procedure esterne
- I processi accedono al monitor invocando una delle sue procedure
- Solo un processo per volta può essere in esecuzione nel monitor (operazioni atomiche, due processi non possono operare nel monitor contemporaneamente)

La sincronizzazione viene ottenuta utilizzando **condition variables**, che sono contenute nel monitor e accessibili solo tramite questo.

Si opera sulle condition variables attraverso due funzioni:

- **cwait(c):** sospende l'esecuzione del processo che chiama la funzione sulla condizione c
- **csignal(c):** riprende l'esecuzione di un processo che era stato bloccato dopo una cwait sulla condizione c

Struttura di un monitor:



- **Scambio di messaggi:**

Quando diversi processi interagiscono l'uno con l'altro devono essere soddisfatte due condizioni fondamentali:

- sincronizzazione, per imporre la mutua esclusione
- comunicazione, per scambiare informazioni

Il **message passing** è un approccio per fornire entrambe queste funzioni. Funziona con distributed systems e shared memory multiprocessor e uniprocessor systems

La funzionalità è fornita normalmente sotto forma di un paio di primitive:

- **send (destination, message)**
- **receive (source, message)**

send e receive sono considerate atomiche, quindi due processi non possono eseguirle contemporaneamente.

Un processo invia informazioni sotto forma di un messaggio a un altro processo designato da una destinazione.

Un processo riceve informazioni eseguendo la primitiva receive, indicando la sorgente del messaggio.

Caratteristiche di design di message systems per comunicazione e sincronizzazione fra processi:



La comunicazione di un messaggio fra due processi implica la sincronizzazione fra i due:

il ricevente non può ricevere un messaggio finché non viene mandato da un altro processo.

Quando una primitiva receive viene eseguita in un processo ci sono due possibilità:

se non c'è nessun messaggio in attesa il processo viene bloccato finché non riceve un messaggio, oppure il processo continua l'esecuzione abbandonando il tentativo di ricevere il messaggio.

Se invece un messaggio è stato inviato precedentemente allora viene ricevuto e l'esecuzione continua.

Vediamo la sincronizzazione.

Blocking Send and Blocking Receive:

- Sia il mittente che il destinatario sono bloccati finché il messaggio non viene consegnato
- Alcune volte viene chiamato rendezvous
- Permette una sincronizzazione stretta fra i processi

Nonblocking Send and Blocking Receive:

- Il mittente continua l'esecuzione, mentre il destinatario è bloccato finché non arriva il messaggio richiesto
- Si possono inviare uno o più messaggi a diverse destinazioni più velocemente possibile
- È la combinazione più utile, ad esempio per un processo che fornisce un servizio o risorse ad altri processi.

Nonblocking Send and Nonblocking Receive:

- Nessun partecipante deve aspettare

Vediamo l'addressing (indirizzamento).

Indirizzamento diretto: la primitiva di send include uno specifico identificatore per il processo destinatario

La primitiva receive può essere gestita in due modi:

- richiede che il processo individui esplicitamente un processo mittente (sending process), efficace per cooperating concurrent processes
- indirizzamento implicito, fra i parametri sorgente della primitiva receive c'è un valore ritornato quando l'operazione di receive è andata a buon fine

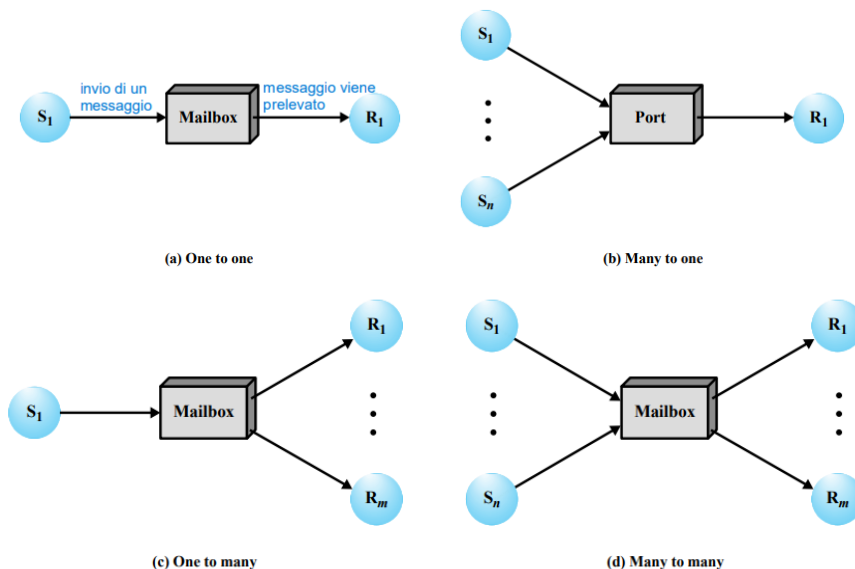
Indirizzamento indiretto: i messaggi sono inviati a una struttura dati condivisa, che consiste in code che possono temporaneamente tenere dei messaggi.

Queste code sono dette **mailboxes**.

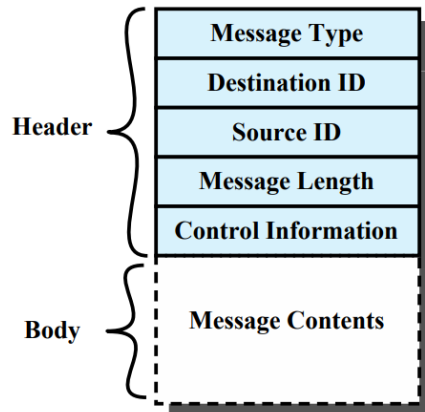
Un processo invia un messaggio alla mailbox e l'altro processo recupera il messaggio da questa.

Permette una maggiore flessibilità nell'uso dei messaggi.

Abbiamo diversi scenari per le mailbox:



Format generale di un messaggio:



A seconda dell'implementazioni ci possono essere più o meno informazioni nell'header.

es. mutua esclusione utilizzando message passing

```

/* program mutualexclusion */
const int n = /* number of processes */;
void P(int i)
{
    message msg; crea messaggio
    while (true) {
        receive (box, msg); riceve il messaggio dalla mailbox, che gli dice che la
                           sezione critica è libera
        /* critical section */;
        send (box, msg); invia messaggio alla mailbox, dicendo che la sezione critica è ora libera
        /* remainder */;
    }
}
void main()
{
    create_mailbox (box); crea una mailbox
    send (box, null); invio un messaggio nella mailbox, è vuoto nel body ma l'header c'è.
    parbegin (P(1), P(2), . . . , P(n)); indica agli altri processi che possono inviarti messaggi
    n processi concorrenti
}
  
```

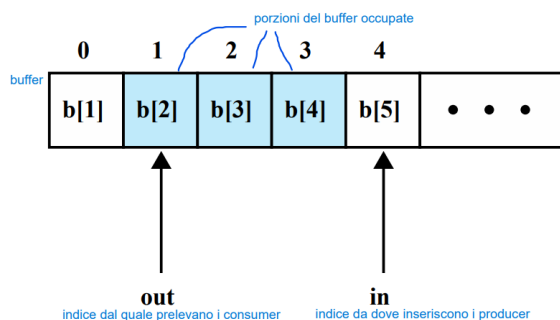
3. PROBLEMI: PRODUCER/CONSUMER, READERS/WRITERS

Problema Producer/Consumer:

uno o più producer generano dati e li piazzano in un buffer. Uno o più consumer prendono uno per volta gli elementi in questo buffer.

Solo un producer o consumer può accedere al buffer in un certo istante di tempo.

Problema: vogliamo assicurarci che il producer non possa aggiungere dati in un buffer pieno, e il consumer non possa rimuovere dati da un buffer vuoto.



Questo problema si può risolvere in diversi modi, ad esempio con un semaforo o con message passing.

- Implementazioni con semaforo

Soluzione sbagliata:

```
/* program producerconsumer */
int n = 0; numero di elementi presenti nel buffer
binary_semaphore s = 1, delay = 0; semaforo binario s inizializzato a 1
                                     semaforo binario delay inizializzato a 0

void producer()
{
    while (true) {
        produce(); genera dati
        semWaitB(s); richiede risorsa
        sezione critica sul buffer {
            append(); inserisce il valore prodotto nel buffer
            n++; dimensione del buffer è incrementata
            if (n==1) semSignalB(delay); se è stato inserito un dato nel buffer, setta il semaforo delay a 1 (segnala che il buffer non è vuoto)
            semSignalB(s); rilascia risorsa
        }
        uso il semaforo delay per bloccare i consumatori quando non c'è nulla.
    }
}

void consumer()
{
    semWaitB(delay); continua solo se delay è =1, cioè se il buffer è non vuoto, altrimenti aspetta
    while (true) {
        semWaitB(s); richiede la risorsa
        sezione critica sul buffer {
            take(); prelevo un elemento
            n--; ho tolto un elemento dal buffer
            semSignalB(s); rilascia la risorsa
            consume(); consuma il dato prelevato
            if (n==0) semWaitB(delay); se ora il buffer è vuoto porta il semaforo delay a 0
        }
    }
}
```

Errore: viene consumato un elemento che non esiste

Producer	Consumer	s	n	Delay
		1	0	0
semWaitB(s)		0	0	0
n++		0	1	0
if (n==1) (semSignalB(delay))		0	1	1
semSignalB(s)		1	1	1
	semWaitB(delay)	1	1	0
	semWaitB(s)	0	1	0
	n--	0	0	0
	semSignalB(s)	1	0	0

Il consumatore viene descheduled prima che possa eseguire il test

if (n==0) semWaitB(delay)

Quindi il producer entra la sezione critica e incrementa n.

	semSignalB(s)	1	0	0
semWaitB(s)		0	0	0
n++		0	1	0
if (n==1) (semSignalB(delay))		0	1	1
semSignalB(s)		1	1	1
	if (n==0) (semWaitB(delay))	1	1	1

Più avanti otteniamo n=-1: stiamo leggendo un elemento invalido.

```
// consumer
semWaitB(s)
take(), n--
semSignalB(s)
consume()
if (n==0) ...
```

semWaitB(s)	0	1	1
n--	0	0	1
semSignalB(s)	1	0	1
if (n==0) (semWaitB(delay))	1	0	0
semWaitB(s)	0	0	0
n--	0	-1	0
semSignalB(s)	1	-1	0

Completivamente:

	Producer	Consumer	s	n	Delay
1			1	0	0
2	semWaitB(s)		0	0	0
3	n++		0	1	0
4	if (n==1) (semSignalB(delay))		0	1	1
5	semSignalB(s)		1	1	1
6		semWaitB(delay)	1	1	0
7		semWaitB(s)	0	1	0
8		n--	0	0	0
9		semSignalB(s)	1	0	0
10	semWaitB(s)		0	0	0
11	n++		0	1	0
12	if (n==1) (semSignalB(delay))		0	1	1
13	semSignalB(s)		1	1	1
14		if (n==0) (semWaitB(delay))	1	1	1
15		semWaitB(s)	0	1	1
16		n--	0	0	1
17		semSignalB(s)	1	0	1
18		if (n==0) (semWaitB(delay))	1	0	0
19		semWaitB(s)	0	0	0
20		n--	0	-1	0
21		semSignalB(s)	1	-1	0

Altra soluzione sbagliata:

```

/* program producerconsumer */
int n = 0; numero di elementi nel buffer
binary_semaphore s = 1, delay = 0; semaforo binario s inizializzato a 1  
semaforo binario delay  
inizializzato a 0

void producer()
{
    while (true) {
        produce(); genera dati
        semWaitB(s); richiede la risorsa
        append(); inserisce dato nel buffer
        n++; dimensione del buffer è incrementata
        if (n==1) semSignalB(delay);
        semSignalB(s); quando è stato inserito il primo elemento nel  
buffer incrementa delay (segnala buffer non vuoto)
        rilascia la risorsa
    }
}

```

```

void consumer()
{
    semWaitB(delay); // continua solo se delay è !=1, cioè se il buffer è non vuoto, altrimenti aspetta
    while (true) {
        semWaitB(s); // richiede la risorsa
        take(); // preleva un elemento
        n--; // dimensione del buffer è decrementata
        consume(); // consuma il dato prelevato
        if (n==0) semWaitB(delay); // se n=0 setta delay=0, segnala che il buffer è vuoto
        semSignalB(s); // rilascia la risorsa
    }
}

```

Deadlock

Errore: porta a deadlock

Possibile soluzione:

Facciamo una copia locale $m=n$ della variabile, così possiamo leggere il valore corretto una volta che lasciamo la sezione critica

```

void consumer()
{
    int m; /* a local variable */
    semWaitB(delay); // continua solo se delay è !=1, cioè se il buffer è non vuoto, altrimenti aspetta
    while (true) {
        semWaitB(s); // richiede la risorsa
        take(); // preleva un elemento dal buffer
        n--; // dimensione del buffer è decrementata
        m = n; // salva il valore di n nella variabile locale m, quindi fuori dalla sezione critica
        semSignalB(s); // rilascia la risorsa
        consume(); // consuma il dato prelevato
        if (m==0) semWaitB(delay); // il controllo lo fa sulla variabile m locale
    }
}

```

Usando i semafori non binari tutto ciò diventa più semplice:

```

/* program producerconsumer */
semaphore n = 0, s = 1;

```

```

void producer()
{
    while (true) {
        produce();
        semWait(s);
        append();
        semSignal(s);
        semSignal(n); // libera il consumer
    }
}

```

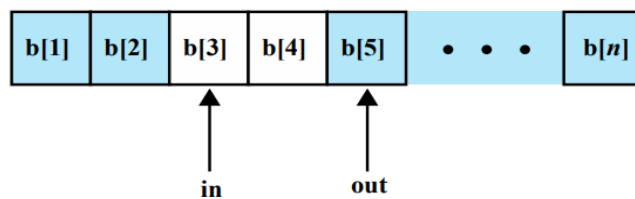
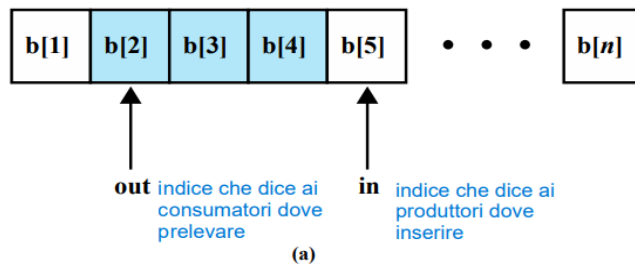
```

void consumer()
{
    while (true) {
        semWait(n);
        semWait(s);
        take();
        semSignal(s);
        consume();
    }
}

```

Questo modello è utilizzabile anche con più produttori e più consumatori

Finite Circular Buffer for the Producer/Consumer Problem:



Abbiamo visto il caso di un buffer infinito. Vediamo quando il buffer ha dimensione finita

```
/* program boundedbuffer */
const int sizeofbuffer = /* buffer size */;
semaphore s = 1, n= 0, e= sizeofbuffer;
```

numero dei posti liberi nel buffer

```
void producer()
{
    while (true) {
        produce();
        semWait(e); /* una posizione è stata occupata nel buffer */
        semWait(s);
        append();
        semSignal(s);
        semSignal(n);
    }
}
```

```
void consumer()
{
    while (true) {
        semWait(n);
        semWait(s);
        take();
        semSignal(s);
        semSignal(e); /* libera la posizione nel buffer */
        consume();
    }
}
```

E se scambio un paio di semWait o semSignal adiacenti nel codice??? E se ho un solo producer o un solo consumer??

```
void producer()
{
    while (true) {
        produce();
        semWait(e);
        semWait(s);
        append();
        semSignal(s);
        semSignal(n);
    }
}
```

```
void consumer()
{
    while (true) {
        semWait(n);
        semWait(s);
        take();
        semSignal(s);
        semSignal(e);
        consume();
    }
}
```

- Implementazione con buffer finito usando Monitor

Monitor:

- i dati locali sono accessibili solo tramite le procedure del monitor
- solo un processo per volta può stare in esecuzione nel monitor.
- il programmatore può decidere quando una procedura in esecuzione nel monitor dovrebbe fermarsi o riprendere a seconda delle condizioni.

Quindi, consume() e produce() non hanno bisogno di sapere come take() e append() sono implementate dentro il monitor.

```
void producer()
{
    char x;
    while (true) {
        produce(x);
        append(x);
    }
}
void consumer()
{
    char x;
    while (true) {
        take(x);
        consume(x);
    }
}
```

```
/* program producerconsumer */
monitor boundedbuffer;
char buffer [N]; /* space for N items */
int nextin, nextout; /* buffer pointers */
int count; /* number of items in buffer */
cond notfull, notempty; /* condition variables for synchronization */

Init: nextin = nextout = count = 0;
```

```

void append (char x)
{
    if (count == N) cwait(notfull);
    buffer[nextin] = x;
    nextin = (nextin + 1) % N;
    count++;
    /* one more item in buffer */
    csignal(notempty);
}

```

Se il buffer è pieno aspetta, notifica il consumer in attesa alla exit (se c'è).

```

void take (char x)
{
    if (count == 0) cwait(notempty);
    x = buffer[nextout];
    nextout = (nextout + 1) % N;
    count--;
    csignal(notfull);
}

```

Se il buffer è vuoto aspetta, noti

Wait if buffer is empty, notifica il producer in attesa alla exit (se c'è).

Anche questo algoritmo supporta molteplici consumer.

- Implementazione con buffer finito usando message passing

```

const int
    capacity = /* buffering capacity */ ;
    null = /* empty message */ ;
int i;
void producer()
{
    message pmsg;
    while (true) {
        receive (mayproduce, pmsg); riceve il messaggio che può produrre
        pmsg = produce(); produce
        send (mayconsume, pmsg); quello che ha prodotto invece che metterlo
                                in un buffare lo invia in un messaggio
    }
}
void consumer()
{
    message cmsg;
    while (true) {
        receive (mayconsume, cmsg); riceve
        consume (cmsg); consuma ciò che ha prodotto il produttore preso dal messaggio
        send (mayproduce, null); ho preso un messaggio dalla mailbox, il
                                produttore può quindi produrre un'altra volta
    }
}
void main()
{
    create_mailbox (mayproduce); crea due mailbox
    create_mailbox (mayconsume);
    for (int i = 1; i <= capacity; i++) send (mayproduce,
null); dice al produttore che può produrre n volte (mandando n messaggi)
    parbegin (producer, consumer);
}

```

Problema Lettore/Scrittore: una area di dati è condivisa fra diversi processi. Alcuni di questi compiono solo lettura nell'area di dati (readers), altri solo la scrittura (writers).

Condizioni che devono essere soddisfatte:

- Qualsiasi numero di readers potrebbe leggere il file simultaneamente
- Solo un writer per volta può scrivere sul file
- Se un writer sta scrivendo sul file, nessun reader può leggere

Soluzione usando semafori:

```
/* program readersandwriters */  
int readcount = 0;  
semaphore x = 1, wsem = 1;
```

Requirements:

- any number of readers may read simultaneously
- only one writer at a time may write
- when a writer is writing, no reader is allowed to read

```
while (true) {  
    semWait (wsem);  
    WRITEUNIT();  
    semSignal (wsem);  
}
```

writer()

- acquire exclusive lock using binary semaphore wsem

```
while (true) {  
    semWait (x);  
    readcount++;  
    if (readcount == 1) semWait (wsem);  
    semSignal (x);  
    READUNIT();  
    semWait (x);  
    readcount--;  
    if (readcount == 0) semSignal (wsem);  
    semSignal (x);  
}
```

reader()

- binary semaphore x to safely update variable readcount
- when there is only one reader (x==1) lock wsem (no writes!)
- once the read is performed, unlock wsem if no reader is active (i.e., readcount==0) => *writers may starve*, extensions needed!